

Browser Artifact Analysis for MFA Login Forensics in Modern Web Environments

Diana Orozek kyzy
Faculty of Computer Science and
Engineering (FINKI)
Ss. Cyril and Methodius University in
Skopje (UKIM)
Skopje, North Macedonia
dianaorozek@gmail.com

Abstract—Web authentication mechanisms generate digital traces that can support forensic investigations despite increasing browser privacy protections. This study investigates login-related web forensic artifacts produced during multi-factor authentication (MFA) workflows in a controlled environment. A locally hosted web application implementing a two-stage authentication process was deployed, and multiple successful and failed MFA login attempts were simulated using Mozilla Firefox on macOS. Browser-side artifacts were analyzed using low-level SQLite-based forensic techniques and correlated with server-side authentication logs.

The results show that traditional authentication artifacts, such as persistent cookies and stored credentials, are largely unavailable in modern MFA-enabled environments. However, indirect evidence remains accessible. Browsing history reveals structured authentication navigation patterns, form history contains residual one-time password (OTP) inputs, and LocalStorage artifacts provide indicators of application interaction and client-side state persistence. These artifacts collectively enable reconstruction of authentication activity in the absence of reusable credentials or long-lived session data.

The study demonstrates the importance of a multi-artifact forensic approach and careful tool selection when analyzing authentication events in contemporary web environments.

Keywords— web forensics, browser forensics, multi-factor authentication, authentication analysis, browser artifacts, LocalStorage, form history, SQLite analysis.

I. INTRODUCTION

A. Web Applications and Authentication

Web applications have become central to modern digital ecosystems, supporting services such as online communication, electronic commerce, education platforms, and cloud-based systems. As reliance on web-based services continues to grow, authentication mechanisms play a critical role in ensuring that access to protected resources is restricted to authorized users. The login process, typically based on username–password authentication combined with session management techniques, represents the primary gateway to web applications and therefore constitutes a major security focal point [1], [2], [9].

Every authentication attempt generates digital traces across multiple system layers, including server-side logs and client-side browser artifacts. These traces may include HTTP requests, timestamps, session identifiers, and browser storage entries. From a cybersecurity and forensic perspective, such artifacts are essential for reconstructing user behavior, investigating suspicious login activity, and supporting post-incident analysis [3][10].

However, modern browsers increasingly implement privacy and security protections that limit the persistence of sensitive authentication data. As a result, traditional artifacts such as stored credentials, persistent cookies, or saved form inputs may be unavailable during forensic investigations. This shift necessitates the examination of alternative and indirect sources of evidence to reconstruct login-related activity.

B. Authentication Threats and Forensic Relevance

The login phase remains a primary target for cyber attacks, including brute-force attempts, credential stuffing, and unauthorized access through session misuse. Reports consistently identify authentication-related incidents as among the most common causes of account compromise in web-based systems [4], [5], [13]. These attacks often generate subtle and distributed traces that are difficult to identify without systematic forensic analysis.

From a digital forensics perspective, login-related incidents require the correlation of multiple evidence sources to reconstruct authentication timelines and user interactions. Browser artifacts, such as browsing history, cookies, and client-side storage, can provide valuable insight into login behavior even when server-side data is incomplete or unavailable [3], [6], [10]. Understanding how modern browser security mechanisms affect the availability of such artifacts is therefore critical for applied cybersecurity investigations.

C. Role of Digital Forensics in Login Analysis

Digital forensics provides structured methodologies for identifying, acquiring, analyzing, and interpreting digital evidence in a forensically sound manner. In the context of web authentication incidents, forensic analysis enables investigators to reconstruct login attempts, identify abnormal access patterns, and assess the effectiveness of security controls [7], [13].

Recent research emphasizes that browser-based evidence plays a crucial role in authentication investigations, particularly when traditional artifacts are limited by encryption or privacy protections [7], [8]. Consequently, forensic practitioners must adopt flexible approaches that consider both direct and indirect indicators of login activity. This study adopts such an approach by focusing on browser-level forensic artifacts generated during authentication interactions.

D. Aim and Objectives

The aim of this study is to analyze and reconstruct login-related user activity through browser artifacts

generated during web authentication, with particular emphasis on how modern browser security and privacy mechanisms influence the availability of forensic evidence.

The objectives of this study are to:

- Simulate authentication activity within a controlled web application environment.
- Collect browser artifacts generated during login attempts using Mozilla Firefox on macOS.
- Analyze multiple browser artifact types, including history databases, cookies, form history, and Web Storage.
- Evaluate the effectiveness and limitations of browser artifacts for reconstructing authentication activity.
- Assess the applicability of SQLite-based forensic analysis tools for login-focused investigations.

Rather than attempting to recover authentication credentials, this study focuses on reconstructing login workflows through indirect browser artifacts, such as browsing history patterns and visit-level metadata, and evaluating their forensic reliability in modern privacy-focused environments.

II. BACKGROUND & RELATED WORK

A. Web Forensics and Browser Artifacts

Web forensics is a subfield of digital forensics that focuses on evidence generated through web-based activity. Because web browsers are the main interface between users and online services, they store a variety of artifacts that are valuable during forensic investigations. These artifacts include browsing history, cached files, cookies, session-related data, download records, and client-side storage. Together, they allow investigators to reconstruct timelines and user behavior related to web application usage.

Recent studies confirm that browser artifacts can persist even when users attempt to limit data retention. Sharma et al. show that modern browsers still retain certain artifacts despite privacy protections, enabling partial reconstruction of user activity [3]. Similarly, studies on private browsing modes demonstrate that browser forensic analysis can recover useful traces across normal and private sessions, although the quantity and quality of evidence may differ [11].

Research also emphasizes that browser artifacts should not be analyzed in isolation. Qazi et al. argue that effective forensic interpretation requires correlation between multiple artifact types to understand user intent and behavior accurately [8]. However, much of the existing literature focuses on general browsing behavior rather than authentication-specific activity, leaving login-phase browser forensics less explored [3], [10]. Authentication-related activity can often only be inferred through correlated browser artifacts, such as visit frequency, access sequences, and client-side storage, rather than direct credential evidence.

B. Authentication Evidence in Modern Web Browsers

Authentication is a critical security process in web applications and a frequent target of cyber-attacks such as brute-force attempts and credential stuffing. Each login attempt produces traces on both the server and client side.

Client-side traces may include browsing history entries, session identifiers, and client-side storage records.

However, modern browsers increasingly restrict the storage of sensitive authentication data. Studies published within the last few years indicate that browsers intentionally limit persistent cookies, form input storage, and credential retention to improve user security and privacy [3], [9], [11]. As a result, investigators are often unable to recover direct evidence such as passwords or long-lived session tokens. Consequently, forensic analysis increasingly relies on indirect indicators of login activity, including browsing history patterns, timestamp correlations, and application-specific client-side storage.

Recent research highlights that authentication-related investigations must therefore rely on indirect indicators of login activity, including access patterns, timestamps, and application-specific client-side storage [6], [8], [10]. Despite recognition of this issue, few studies provide practical demonstrations of how such indirect artifacts can be used to reconstruct login activity in modern environments [3], [8].

C. Web Server Logs and Digital Evidence

Web server logs are widely recognized as a primary source of evidence in web forensic investigations. They contain structured records of HTTP requests, response codes, timestamps, and client IP addresses, making them useful for identifying login attempts and abnormal access behavior.

Several recent studies highlight the effectiveness of log analysis in detecting authentication attacks and suspicious access patterns [4], [12]. Tools such as ELK Stack and Splunk are commonly used to process large volumes of log data and identify trends or anomalies. However, server logs alone provide limited insight into how users interact with login interfaces at the browser level [10].

Recent literature notes that server-side evidence should be combined with browser-side artifacts to provide a more complete view of authentication events [7], [10]. This combined perspective is particularly important when browser-side artifacts provide only indirect evidence of authentication activity. Nevertheless, empirical studies that integrate both sources in login-focused forensic investigations remain limited, representing an important gap [3], [8].

D. Challenges in Web Artifact Analysis

Web artifact analysis faces significant challenges due to the rapid evolution of browser technologies and security features. Modern browsers frequently update storage formats, introduce encryption, and restrict access to sensitive data, which directly affects forensic analysis. Raza et al. demonstrate that browser artifact lifecycles vary across browser versions and operating systems, complicating consistent evidence of extraction [6].

Privacy-enhancing features such as private browsing modes, encrypted databases, and restricted form history storage further reduce the availability of traditional forensic artifacts. Recent studies emphasize that these protections limit the effectiveness of automated forensic tools, often requiring manual or low-level analysis techniques [3], [11].

Additionally, many forensic tools are optimized for specific platforms or artifact types and may not fully support newer storage mechanisms such as Web Storage. This highlights the importance of evaluating forensic tools in modern environments and understanding their limitations when applied to authentication-related investigations.

E. Forensic Frameworks and Methodologies

A wide range of digital forensic tools has been proposed for web investigations. General-purpose forensic tools such as Autopsy support the extraction of browser artifacts and provide timeline and reporting features. Log-based tools, including ELK Stack and Splunk, are effective for analyzing server logs and detecting large-scale authentication attacks.

Recent comparative studies indicate that while automated tools are useful for high-level analysis, they often fail to extract detailed browser artifacts related to authentication workflows [3], [8]. In such cases, manual analysis of browser databases using SQLite-based tools can provide deeper insight into client-side evidence. Such low-level analysis is especially relevant for login-phase investigations, where modern privacy mechanisms limit the effectiveness of automated extraction tools.

However, the literature shows limited evaluation of how these tools perform specifically in login-phase investigations under modern browser privacy constraints. This lack of focused comparison represents a gap that motivates further study.

F. Identified Gaps and Research Motivation

Based on the reviewed literature, several gaps can be identified. First, although browser forensics is well studied, fewer works focus specifically on login-phase investigations and authentication-related artifacts. Second, many studies analyze server logs and browser artifacts separately, with limited integration of both perspectives. Third, modern browser privacy protections significantly reduce the availability of traditional authentication artifacts, yet practical studies addressing this limitation remain scarce.

This study addresses these gaps by focusing on login-related browser artifacts generated during authentication workflows in a modern browser environment. Instead of attempting to recover sensitive credentials, the study evaluates how indirect artifacts, such as browsing history and client-side storage, can support forensic reconstruction. In addition, it provides a comparative discussion of forensic tools, highlighting their effectiveness and limitations in contemporary web investigations.

III. METHODOLOGY

The methodology for this study outlines the process used to generate, collect, and analyze login-phase digital evidence from web applications. The primary objective was to simulate real-world authentication activity and evaluate the effectiveness of a selected digital forensic tool in extracting and interpreting artifacts relevant to login events. Rather than attempting to recover authentication credentials, the methodology emphasizes reconstruction of login workflows through indirect client-side and server-side artifacts.

A. Experimental Environment

The experimental software environment and forensic tools used in this study are summarized in Table I. The environment was intentionally designed to reflect a realistic, modern web usage scenario while maintaining experimental control. The experiments were conducted on macOS Tahoe (version 26.2), a contemporary operating system that incorporates strong built-in security and privacy mechanisms, which directly influence the persistence and accessibility of browser forensic artifacts.

Mozilla Firefox (version 120+) was selected as the web browser due to its widespread adoption and extensive coverage in prior browser forensic research. Firefox stores key artifacts such as browsing history, cookies, form history, and Web Storage in SQLite-based databases, making it particularly suitable for low-level forensic analysis.

The web application used in the experiment was implemented in Python (version 3.9) using the Flask web framework (version 3.1). Flask was chosen because of its lightweight architecture and explicit control over authentication logic, allowing precise simulation of login workflows, including successful and failed authentication attempts, as well as logout events.

DB Browser for SQLite (version 3.13) was employed as the primary forensic analysis tool to manually inspect browser artifact databases. This tool enables direct querying and examination of SQLite structures, which is essential when automated forensic tools are limited by modern browser encryption and privacy protections.

Finally, the web application was hosted locally on the same system using a localhost server environment (127.0.0.1). This configuration ensured that all network interactions, server logs, and browser artifacts were generated within a controlled environment, eliminating external network variables and allowing reliable correlation between server-side authentication events and client-side browser evidence.

TABLE I. EXPERIMENTAL TOOLS AND ENVIRONMENT

Component	Description	Version
Operating System	macOS Tahoe	26.2
Web Browser	Mozilla Firefox	120+
Programming Language	Python	3.9
Web Framework	Flask	3.1
Forensic Tool	DB Browser for SQLite	3.13
Server Environment	Localhost (127.0.0.1)	

The selected operating system and browser represent a modern, privacy-focused environment, which directly influences the availability and structure of browser forensic artifacts.

B. Web Application Design and Login Simulation

To simulate realistic modern authentication activity, a web application implementing a multi-factor authentication (MFA)-based login workflow was developed using Flask.

The application consisted of an initial credential verification stage (username and password), a secondary MFA verification stage using one-time passwords (OTP), a protected dashboard page, and a logout function. The authentication logic was intentionally kept simple to prioritize the generation and examination of forensic artifacts rather than application-level security hardening.

The authentication workflow followed a two-step process. After submitting valid username–password credentials, users were redirected to an MFA verification page where a time-limited OTP was required to complete authentication. Access to the protected dashboard was granted only after successful OTP verification. Failed password attempts, failed OTP submissions, expired OTPs, and successful MFA logins were all deliberately generated to reflect common real-world authentication behavior in systems protected by MFA.

The application recorded authentication outcomes through server-side logging mechanisms, capturing timestamps and state transitions for each stage of the login process. Logged events included initial credential validation, OTP generation, OTP verification (successful or failed), OTP expiration, and logout actions. These interactions were designed to produce a diverse set of authentication-related artifacts across both browser-side and server-side sources.

This MFA-focused approach ensured that browser-side evidence (such as browsing history and client-side storage) and server-side evidence (detailed MFA logs) were generated under conditions representative of contemporary authentication systems.

C. Evidence Collection

After completing the MFA-based authentication interactions, the Firefox browser was fully closed to ensure that volatile session data and in-memory state were written to persistent storage. A forensic copy of the Firefox profile directory was then created from the macOS file system. All subsequent analysis was performed on the copied profile to preserve the integrity of the original evidence.

Collected browser artifacts were examined at the database level to ensure visibility into stored structures and values not accessible through standard browser interfaces. The following client-side artifacts were collected and analyzed:

- Browsing history (places.sqlite)
- Cookie storage (cookies.sqlite)
- Form history (formhistory.sqlite)
- Web Storage (storage/default)

In parallel, server-side MFA-related logs generated by the Flask application were collected. These logs included records of OTP generation, verification attempts, expiration events, resend actions, and session state transitions associated with the MFA workflow. The combination of browser-side and server-side evidence enabled correlation between user actions in the browser and authentication events recorded by the application backend.

D. Forensic Tools and Analysis Approach

DB Browser for SQLite was used as the primary forensic analysis tool to examine browser artifact databases at a low

level. Structured queries were executed against browser history and client-side storage databases to identify authentication-related activity associated with the MFA workflow. Analysis focused on login and verification endpoints, access sequencing between password and OTP stages, visit frequency, and timestamp-based correlation to support reconstruction of MFA authentication flows.

The effectiveness of the selected tools was evaluated within the context of macOS and Firefox, both of which implement strong privacy protections that limit the persistence of sensitive authentication data. As a result, traditional artifacts such as stored credentials, OTP values, or long-lived session cookies were not expected to be recoverable. Instead, the analysis emphasized indirect indicators of MFA activity, including browsing history patterns, page transition sequences, and client-side storage entries. These artifacts were analyzed collectively rather than in isolation to infer authentication behavior under modern privacy constraints.

Server-side MFA logs were used to validate and contextualize findings from browser analysis. This combined approach enabled reconstruction of multi-factor authentication activity despite the absence of traditional login artifacts.

IV. PRACTICAL ANALYSIS - LOGIN-FOCUSED WEB FORENSICS

A. Practical Setup and Evidence Generation

A controlled test environment was established to generate authentication-related digital evidence associated with **multi-factor authentication (MFA)** workflows. A web application implementing a two-stage authentication process—initial username–password verification followed by one-time password (OTP)–based MFA—was deployed locally using a Flask-based backend. The application was accessed through Mozilla Firefox running on macOS, ensuring that browser artifacts were generated under realistic user interaction conditions.

To simulate authentication activity, multiple MFA login scenarios were performed, including failed credential submissions, successful password verification followed by failed or expired OTP entries, and fully successful MFA-authenticated sessions. The login (/login) and MFA verification endpoints were accessed repeatedly to emulate normal user behavior as well as repeated authentication attempts commonly observed in MFA-protected systems.

After completion of these interactions, the Firefox browser was fully closed to ensure that volatile session data was committed to persistent storage. A forensic copy of the Firefox profile directory was then created from the macOS path:

~/Library/Application Support/Firefox/Profiles/

The copied profile was stored in a separate analysis directory to preserve evidential integrity. All subsequent analysis was conducted exclusively on the copied data to prevent modification of original artifacts. Login-related activity was reconstructed using relative event ordering and cross-artifact correlation rather than absolute timestamp values, which may vary in controlled experimental environments.

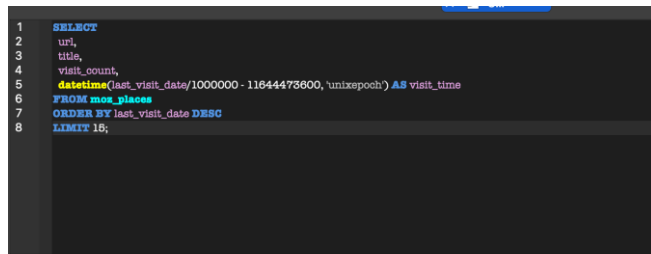
B. Browser History Analysis (places.sqlite)

The first stage of forensic analysis focused on browser history artifacts stored within the places.sqlite database. This SQLite database maintains records of visited URLs and associated metadata, making it a primary source for reconstructing user navigation behavior during authentication workflows.

The database was examined using DB Browser for SQLite, and targeted SQL queries were executed against the moz_places to identify access to the login endpoint and related MFA authentication routes. Firefox timestamps, stored as microseconds since January 19, 1601, were converted into human-readable format to enable temporal correlation of authentication events.

As shown in Fig. 1, the login endpoint (<http://127.0.0.1:5000/login>) appears with a high visit count relative to other application routes, indicating repeated interactions with the initial authentication interface. Additional entries corresponding to MFA verification, protected dashboard access, and logout endpoints further support reconstruction of a complete multi-step authentication workflow rather than isolated page visits.

By correlating URL visit counts with converted timestamp values, clusters of access occurring within short time intervals were identified. These clustered patterns are consistent with repeated authentication attempts followed by server-side redirects between credential submission, MFA verification, and session establishment stages. Such behavior provides strong indirect evidence of login activity in an MFA-enabled system rather than casual or exploratory navigation.



```
1 SELECT
2 url,
3 title,
4 visit_count,
5 datetime(last_visit_date/1000000 - 11644473600, 'unixepoch') AS visit_time
6 FROM moz_places
7 ORDER BY last_visit_date DESC
8 LIMIT 15;
```

	url	title	visit_count	visit_time
1	http://127.0.0.1:5000/login	NULL	13	1687-01-31 23:55:49
2	http://127.0.0.1:5000/verify	NULL	3	1687-01-31 23:06:51
3	http://127.0.0.1:5000/logout	NULL	3	1687-01-31 23:01:12
4	http://127.0.0.1:5000/dashboard	NULL	5	1687-01-31 22:56:54
5	http://127.0.0.1:5000/	404 Not Found	1	1687-01-31 22:50:57
6	https://accounts.firefox.com/?...	Mozilla accounts	1	1687-01-19 16:22:30
7	https://accounts.firefox.com/?...	Mozilla accounts	1	1687-01-19 16:22:29
8	https://www.mozilla.org/about/	NULL	0	NULL
9	https://www.mozilla.org/contribute/	NULL	0	NULL
10	https://support.mozilla.org/kb/customize-firefox-controls...	NULL	0	NULL
11	https://support.mozilla.org/products/firefox	NULL	0	NULL

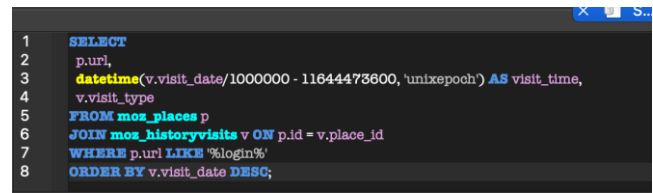
Fig. 1. Login URLs (places.sqlite)

To analyze visit-level behavior in greater detail, browsing history entries were joined with individual visit records from the moz_historyvisits table. This enabled reconstruction of a fine-grained timeline of access to the login and MFA verification endpoints, including visit timestamps and visit types.

As shown in Fig. 2, repeated accesses to the login endpoint occur within short time intervals and are associated

with different visit types, such as direct navigation, redirects, and reload events. These visit patterns reflect structured interaction with the authentication interface, including transitions from credential submission to MFA verification and onward to protected resources.

The presence of multiple visit types within a compact time window is consistent with MFA authentication workflows in which successful credential submission triggers server-side verification, OTP validation, and subsequent redirects. Although explicit credentials, OTP values, and session tokens are not persisted by the browser, these browsing history patterns provide strong indirect forensic evidence of MFA-based authentication activity.



```
1 SELECT
2 p.url,
3 datetime(v.visit_date/1000000 - 11644473600, 'unixepoch') AS visit_time,
4 v.visit_type
5 FROM moz_places p
6 JOIN moz_historyvisits v ON p.id = v.place_id
7 WHERE p.url LIKE '%login%'
8 ORDER BY v.visit_date DESC;
```

	url	visit_time	visit_type
1	http://127.0.0.1:5000/login	1687-01-31 23:55:49	1
2	http://127.0.0.1:5000/login	1687-01-31 23:01:12	6
3	http://127.0.0.1:5000/login	1687-01-31 22:58:00	9
4	http://127.0.0.1:5000/login	1687-01-31 22:44:43	2
5	http://127.0.0.1:5000/login	1687-01-31 22:35:36	6
6	http://127.0.0.1:5000/login	1687-01-31 22:32:18	2
7	http://127.0.0.1:5000/login	1687-01-31 22:28:57	2
8	http://127.0.0.1:5000/login	1687-01-31 22:20:49	2
9	http://127.0.0.1:5000/login	1687-01-31 22:18:10	2
10	http://127.0.0.1:5000/login	1687-01-31 22:15:24	2
11	http://127.0.0.1:5000/login	1687-01-31 21:26:11	2
12	http://127.0.0.1:5000/login	1687-01-19 15:24:10	2
13	http://127.0.0.1:5000/login	1687-01-19 15:23:16	6
14	http://127.0.0.1:5000/login	1687-01-19 15:22:49	2

Fig. 2. Correlation of browsing history and visit-level events illustrating sequential login activity and redirect behavior.

C. Cookie Analysis (cookies.sqlite)

The next phase of analysis examined the cookies.sqlite database, which stores persistent browser cookies. Cookies are commonly used to maintain authenticated sessions following successful login events.

Inspection of the database structure confirmed the presence of the standard moz_cookies table. However, no persistent authentication cookies associated with the test web application were identified. This suggests that the application did not rely on long-lived client-side cookies for session management, or that session cookies were configured to be non-persistent.

The absence of authentication cookies reflects modern web security practices, where sensitive session data is often managed server-side or stored in transient memory rather than persistent browser storage. This finding highlights a limitation of cookie-based forensic analysis when investigating contemporary web applications.

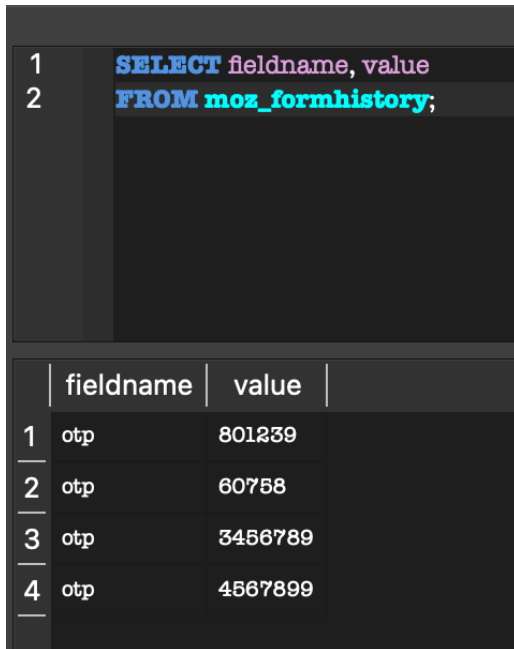
D. Form History Analysis (*formhistory.sqlite*)

To identify potential traces of user input generated during authentication attempts, the *formhistory.sqlite* database was analyzed. This database is used by Firefox to store selected form input values entered by users, such as text fields and numeric inputs, depending on browser configuration, field attributes, and interaction context.

Examination of the *moz_formhistory* table revealed stored values associated with one-time password (OTP) input fields used during the MFA verification stage. As shown in Fig. 3, multiple numeric OTP values were recovered, indicating that transient authentication inputs may be cached by the browser under specific conditions. No usernames or static passwords were identified within the database, reflecting modern browser protections that prevent persistent storage of primary authentication credentials.

From a forensic perspective, the recovered OTP values represent short-lived but sensitive client-side artifacts directly associated with multi-factor authentication activity. Although these OTPs cannot be reused for authentication due to their limited validity period, their presence provides strong temporal and contextual evidence of MFA interaction.

Consequently, form history artifacts are primarily valuable for reconstructing authentication timelines and confirming user participation in MFA workflows, rather than for recovering reusable credentials. When correlated with browsing history and server-side MFA logs, these artifacts significantly strengthen the reconstruction of multi-factor authentication events.



```
1 SELECT fieldname, value
2 FROM moz_formhistory;
```

	fieldname	value
1	otp	801239
2	otp	60758
3	otp	3456789
4	otp	4567899

Fig. 3. Extracted OTP values stored in Firefox *formhistory.sqlite*, demonstrating residual client-side artifacts related to MFA input fields.

E. LocalStorage Analysis (*storage/default*)

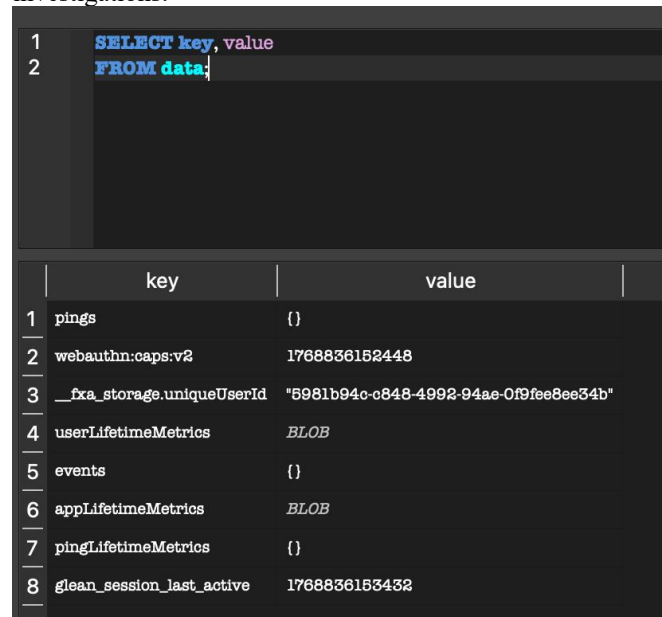
Given the absence of persistent authentication cookies and the limited availability of reusable credentials in form history artifacts, the analysis was extended to client-side Web Storage mechanisms. In Mozilla Firefox, LocalStorage data is stored within the *storage/default* directory of the browser profile and is organized by web origin.

A directory corresponding to the locally hosted web application (<http://127.0.0.1:5000>) was identified,

confirming that the application made use of client-side storage. The *data.sqlite* database located within the *ls* subdirectory was examined using DB Browser for SQLite to analyze stored key-value pairs associated with the application.

As shown in Fig. 4, multiple LocalStorage entries related to application usage were recovered, including persistent identifiers and session-related metadata. These artifacts indicate repeated interaction with the web application and the persistence of client-side state across browser sessions. Although no explicit authentication credentials, OTP values, or reusable session tokens were identified, the presence and continuity of LocalStorage entries provide indirect forensic evidence of authentication-related activity.

This finding is particularly significant in the context of modern MFA-enabled web applications, which increasingly rely on client-side storage mechanisms such as LocalStorage to maintain application state while minimizing the persistence of sensitive authentication data. As a result, relevant forensic artifacts may be shifted away from traditional cookie-based storage, highlighting the growing importance of Web Storage analysis for reconstructing authentication behavior in contemporary web forensic investigations.



```
1 SELECT key, value
2 FROM data;
```

	key	value
1	pings	{}
2	webauthn:caps.v2	1768836152448
3	_fxa_storage.uniqueUserId	"5981b94c-c848-4992-94ae-0f9fee8ee34b"
4	userLifetimeMetrics	BLOB
5	events	{}
6	appLifetimeMetrics	BLOB
7	pingLifetimeMetrics	{}
8	glean_session_last_active	1768836153432

Fig. 4. Representative LocalStorage key-value entries extracted from Firefox *data.sqlite*, demonstrating persistent client-side application state without stored authentication credentials.

F. Summary of Practical Findings

The practical analysis demonstrates that login-related forensic evidence in modern web applications is distributed across multiple browser artifact types, each offering different levels of evidentiary value. Browser history artifacts reliably documented repeated access to authentication interfaces and associated workflow transitions, enabling reconstruction of user navigation during login attempts. In contrast, cookie artifacts provided minimal evidential value due to contemporary security and privacy mechanisms that limit the persistence of authenticated session data.

Form history analysis revealed residual one-time password (OTP) input values associated with multi-factor

authentication (MFA) interactions, representing short-lived but forensically significant client-side artifacts. Additionally, LocalStorage artifacts provided indirect indicators of application usage and client-side state persistence, supporting inference of authentication-related activity even in the absence of reusable credentials or long-lived session tokens.

Collectively, these findings highlight the necessity of a multi-artifact forensic approach when investigating authentication events in modern web environments. By correlating browser history, form history, and client-side storage artifacts, it is possible to reconstruct login-phase behavior in MFA-enabled systems despite the deliberate minimization of persistent authentication data.

G. Practical Contribution

This practical investigation illustrates how browser-level forensic analysis can be used to reconstruct authentication activity even when direct evidence such as credentials or session cookies is unavailable. By combining history, storage, and contextual analysis, investigators can derive meaningful conclusions regarding login behavior while respecting the constraints imposed by modern security architectures.

H. Multi-Factor Authentication (MFA) Artifact Analysis

The implemented multi-factor authentication (MFA) mechanism generated additional forensic artifacts beyond those observed in traditional single-factor login workflows. Each successful username–password authentication triggered server-side generation of a time-limited one-time password (OTP), which was stored transiently within the application session together with its generation timestamp.

OTP verification attempts, expiration events, and regeneration requests were systematically recorded within server-side log files, creating a detailed and ordered record of authentication behavior. In contrast to browser-side artifacts, OTP values and verification state were not persistently stored within browser databases, reflecting modern security practices that intentionally avoid client-side persistence of sensitive MFA data.

From a forensic perspective, these MFA-related server logs substantially enhance the ability to reconstruct authentication events. They enable precise identification of failed authentication attempts, delayed or incorrect OTP submissions, repeated OTP regeneration, and successful MFA completion. When correlated with browser-side artifacts such as browsing history and form history, server-side MFA logs provide explicit evidence of authentication state transitions that are otherwise difficult to infer from client-side data alone.

V. CONCLUSION AND FUTURE WORK

This study examined the forensic investigation of login-related web activity with a particular focus on **multi-factor authentication (MFA) workflows**, using browser and server-side artifacts generated during authentication processes. A controlled experimental environment was employed to simulate successful and failed MFA login attempts in a web application, and browser-side evidence was analyzed using low-level SQLite-based forensic techniques.

The findings demonstrate that modern browsers significantly restrict the availability of traditional

authentication artifacts, such as persistent cookies and stored credentials, especially in MFA-enabled environments. Despite these constraints, meaningful forensic evidence can still be recovered through indirect artifacts. Browser history reliably documented authentication-related navigation behavior, form history revealed residual one-time password (OTP) inputs, and LocalStorage artifacts provided valuable indicators of application interaction and client-side state persistence, even in the absence of reusable authentication credentials or long-lived session tokens.

The study further highlights the importance of selecting forensic tools and methodologies appropriate to the operating system, browser, and authentication architecture under investigation. Automated forensic tools may fail to extract relevant evidence due to modern privacy protections, whereas low-level database analysis enables deeper insight into browser artifacts that are otherwise overlooked. This underscores the need for flexible forensic methodologies and awareness of tool limitations when analyzing contemporary web authentication systems.

Several limitations of this study should be acknowledged. The experiments were conducted using a single operating system, browser, and a simplified MFA implementation within a controlled environment. Additionally, large-scale attack scenarios, multiple concurrent users, and real-world cloud-based authentication services were not examined. These factors limit the direct generalization of the results to more complex production environments.

Future work may extend this research by analyzing MFA artifacts across multiple browsers and operating systems, incorporating real-world authentication frameworks, and performing deeper correlation between browser-side evidence and server-side authentication logs. Further research may also explore the application of machine learning techniques to automate the detection of anomalous authentication behavior by correlating multi-source forensic evidence across both client and server domains.

REFERENCES

- [1] OWASP Foundation, Authentication Cheat Sheet, 2023.
- [2] Z. Durumeric et al., “The Security Impact of HTTPS Interception,” in Proc. IEEE Symp. Security and Privacy, 2020.
- [3] R. R. C. Sharma et al., “Advancing Web Browser Forensics: Critical Evaluation of Emerging Tools and Techniques,” SN Computer Science, 2025.
- [4] Verizon, Data Breach Investigations Report, 2023.
- [5] European Union Agency for Cybersecurity (ENISA), Threat Landscape for Credential-Based Attacks, 2022.
- [6] A. Raza et al., “Forensic Analysis of Web Browsers Lifecycle: A Case Study,” Journal of Information Security and Applications, 2024.
- [7] A. Dimitriadis and N. Ivezic, “D4I: A digital forensics framework for reviewing and investigating cyber-attacks,” Digital Investigation, vol. 44, 2023.
- [8] E. U. H. Qazi, A. Ahmed, and M. A. Shah, “Examining the behavior of web browsers using popular forensic tools,” International Journal of Digital Crime and Forensics, vol. 16, no. 1, 2024.
- [9] NIST, Digital Identity Guidelines: Authentication and Lifecycle Management (SP 800-63B), 2022.
- [10] NIST, Guide to Integrating Forensic Techniques into Incident Response (SP 800-86), 2006.
- [11] J. Lim et al., “SOK: On the Analysis of Web Browser Security,” arXiv preprint, 2021.

- [12] European Union Agency for Cybersecurity (ENISA), Cybersecurity Threat Landscape, 2023.
- [13] NIST, Guide to Integrating Forensic Techniques into Incident Response (SP 800-86), 2022.